SHOPSPHERE

RTU Advanced Java / Java Enterprise E-Commerce Project

Complete Student Project Development Guide

This guide is based on the RTU syllabus provided: Swing, MVC Architecture, Applets, JDBC, java.net networking, Client-Server, RMI, RMI Registry, Naming/JNDI, Serialization, Internationalization, J2EE architecture, N-tier applications, HTTP, Web Containers, Servlets, Sessions, Events, Filters, JSP, EL, Tag Extensions, JSP Fragments, Tag Files and JSTL.

1. Project Overview

ShopSphere is a complete Java E-Commerce Management System designed as a final practical project. The project is deliberately structured so that students can demonstrate the major RTU Advanced Java topics inside one coherent application.

The main customer experience is a modern web store. Additional desktop, networking and distributed-service modules demonstrate the syllabus topics that do not naturally belong inside a browser application.

2. Project Objectives

- Build a complete E-Commerce application using Java and MVC/layered architecture.
- Use Swing for a desktop administration module.
- Use JDBC and MySQL for persistent data.
- Implement Java networking with Socket and ServerSocket.
- Implement RMI and the RMI registry for a remote inventory service.
- Demonstrate object serialization.
- Apply J2EE, HTTP, web-container and N-tier concepts.
- Build the customer application using Jakarta Servlet and JSP on Apache Tomcat.
- Use sessions, cookies, listeners and filters in meaningful features.
- Use JSP EL, JSTL, JSP fragments and tag files.
- Provide an explicit RTU syllabus-to-project mapping.

3. RTU Syllabus Mapping

RTU Topic	Project Implementation
Swing	Desktop Admin/Management application
Swing components	Login, forms, buttons, text fields, checkboxes, radio buttons, tables
MVC Architecture	Swing MVC and web MVC/layered architecture
Applets	Short historical/concept demonstration; not a dependency of the modern app

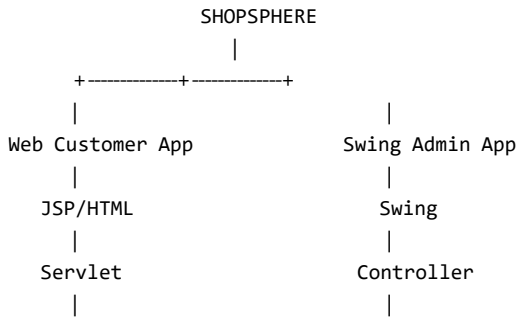
JDBC / java.sql	DAO layer for users, products, carts and orders
JDBC Driver	MySQL Connector/J
java.net	Socket/ServerSocket inventory client-server module
Client and Server	ShopSphere Inventory Server + Java client
Content/Protocol Handlers	Small URL/URLConnection demonstration
RMI Architecture	Remote inventory/order service
RMI Registry	Registry-based remote service lookup
Naming/JNDI	RMI naming plus JNDI resource lookup demonstration
Serialization	Transfer Product/Order objects in network module
Internationalization	Locale, ResourceBundle and formatted values
J2EE architecture	Complete enterprise-style web application
N-tier architecture	Presentation → Controller → Service → DAO → Database
HTTP/Web application	Browser ↔ Tomcat request/response
Web container	Apache Tomcat
Servlet	Customer/Admin request controllers
Servlet lifecycle	init/service/doGet/doPost/destroy
ServletConfig/ServletContext	Servlet-specific and application-wide configuration
Request/Response	Forms, search, login, checkout and orders
Session handling	Login, cart, checkout and user state
Event handling	ServletContextListener and HttpSessionListener
Filters	AuthenticationFilter and AdminFilter
JSP architecture/lifecycle	Dynamic web views
JSP elements	Declaration, scriptlet, expression and comments for syllabus demo
Expression Language	${product.name}, ${sessionScope.user.name}
Tag Extensions/Handlers	Reusable product/rating components
JSP Fragments	Header, navbar, footer and sidebar

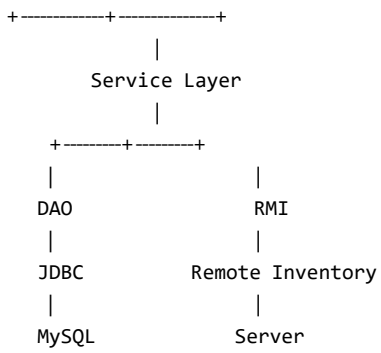
Tag Files	Reusable product card/rating tags
JSTL Core	c:forEach, c:if, c:choose, c:set
JSTL XML	Separate XML product-feed demonstration
JSTL SQL	Separate syllabus demonstration; main app uses DAO/JDBC
JSTL Functions	String/list operations in JSP

4. Technology Stack

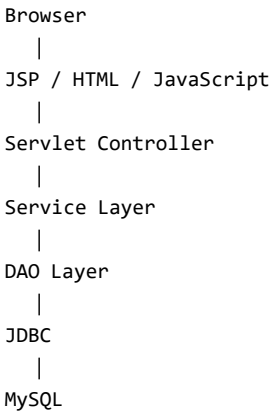
Layer	Technology
Language	Java
Web	Jakarta Servlet + JSP
View	JSP + EL + JSTL + HTML5 + CSS3 + JavaScript + Bootstrap
Database	MySQL
Database access	JDBC / MySQL Connector/J
Build	Maven
Server	Apache Tomcat 11
Desktop	Java Swing
Networking	java.net Socket / ServerSocket
Distributed module	Java RMI
IDE	VS Code / IntelliJ IDEA / Eclipse

5. Overall Architecture





6. N-Tier Web Architecture



The main web application should keep presentation, request processing, business logic and database access separate.

7. Main Functional Modules

Customer Module

- Registration
- Login/logout
- Product browsing
- Search, filter and sort
- Product details
- Shopping cart
- Wishlist
- Address management
- Coupons
- Checkout

- Mock payment
- Order placement
- Order history and tracking
- Profile
- Reviews and ratings

Admin Module

- Admin login
- Dashboard
- Product CRUD
- Category CRUD
- Inventory
- Customer list
- Order management
- Coupon management
- Low-stock view
- Basic sales summary

8. Database Design

Create the database:

```
CREATE DATABASE shopsphere;
```

Table	Important Columns
users	user_id, name, email, password, mobile, role, status, created_at
addresses	address_id, user_id, address_line, city, state, pincode, address_type
categories	category_id, name, description, status
products	product_id, category_id, name, brand, description, price, discount, stock, image_url, status, created_at
cart	cart_id, user_id, product_id, quantity

wishlist	wishlist_id, user_id, product_id, created_at
orders	order_id, user_id, address_id, total_amount, discount, payment_method, payment_status, order_status, created_at
order_items	order_item_id, order_id, product_id, quantity, price
payments	payment_id, order_id, payment_method, transaction_reference, amount, payment_status, created_at
reviews	review_id, user_id, product_id, rating, review_text, created_at
coupons	coupon_id, code, discount_type, discount_value, minimum_order, maximum_discount, expiry_date, status
coupon_usage	usage_id, coupon_id, user_id, order_id, used_at

Database Relationships

```
USER
 |-- ADDRESS
 |-- CART ——— PRODUCT
 |-- WISHLIST —— PRODUCT
 |-- ORDER
 |     `-- ORDER_ITEM — PRODUCT
 `-- REVIEW ——— PRODUCT

CATEGORY ——— PRODUCT
ORDER ——————— PAYMENT
COUPON ——————— COUPON_USAGE
```

9. Recommended Project Structure

```
ShopSphere/
|-- pom.xml
|-- database/
|   `-- shopsphere.sql
|-- swing-admin/
|   `-- src/
|       |-- model/
|       |-- view/
|       |-- controller/
|       `-- dao/
|-- network/
|   |-- client/
|   |-- server/
|   `-- model/
|-- rmi/
|   |-- remote/
|   |-- server/
```

```
|   `-- client/
`-- web/
    `-- src/main/
        |-- java/com/shopsphere/
        |   |-- controller/
        |   |-- service/
        |   |-- dao/
        |   |-- model/
        |   |-- filter/
        |   |-- listener/
        |   `-- util/
        `-- webapp/
            |-- index.jsp
            |-- login.jsp
            |-- register.jsp
            |-- products.jsp
            |-- product-details.jsp
            |-- cart.jsp
            |-- checkout.jsp
            |-- orders.jsp
            |-- profile.jsp
            |-- wishlist.jsp
            |-- admin/
            |-- fragments/
            |-- WEB-INF/tags/
            |-- css/
            |-- js/
            `-- images/
```

10. Model Classes

- User
- Address
- Category
- Product
- CartItem
- WishlistItem
- Order
- OrderItem
- Payment
- Review
- Coupon

```
public class Product {
    private int productId;
```

```
    private int categoryId;
    private String name;
    private String brand;
    private String description;
    private double price;
    private double discount;
    private int stock;
    private String imageUrl;
    private boolean status;
    // constructors, getters and setters
}
```

11. JDBC and DAO Layer

Create DAO classes such as UserDAO, ProductDAO, CategoryDAO, CartDAO, OrderDAO, OrderItemDAO, ReviewDAO and CouponDAO. Use Connection, PreparedStatement, ResultSet, DriverManager and SQLException.

```
PreparedStatement ps = connection.prepareStatement(
    "SELECT * FROM products WHERE product_id = ?"
);
ps.setInt(1, id);
ResultSet rs = ps.executeQuery();
```

Do not concatenate user input into SQL strings. Use PreparedStatement.

12. Service Layer

```
ProductService
    -> ProductDAO

OrderService
    -> OrderDAO
    -> OrderItemDAO
    -> ProductDAO

CheckoutService
    -> CartDAO
    -> CouponDAO
    -> OrderDAO
    -> PaymentDAO
```

Business rules belong in the service layer, not inside JSP pages.

13. Web Pages

Page	Purpose
index.jsp	Home page

login.jsp	Customer/admin login
register.jsp	Registration
products.jsp	Listing/search/filter
product-details.jsp	Product information
cart.jsp	Shopping cart
checkout.jsp	Address, coupon and payment
order-success.jsp	Confirmation
orders.jsp	Order history
order-details.jsp	Order details/tracking
profile.jsp	Profile
wishlist.jsp	Wishlist
admin/dashboard.jsp	Admin dashboard
admin/products.jsp	Product management
admin/categories.jsp	Category management
admin/orders.jsp	Order management
admin/users.jsp	Customer management

14. Servlet Controllers

```
LoginServlet
RegisterServlet
LogoutServlet
ProductServlet
CartServlet
CheckoutServlet
OrderServlet
WishlistServlet
ReviewServlet

AdminLoginServlet
AdminProductServlet
AdminCategoryServlet
AdminOrderServlet
AdminUserServlet
```

Servlet Lifecycle

```
Loading -> Instantiation -> init()
```

```
-> service()
-> doGet()/doPost()
-> destroy()
```

ServletConfig and ServletContext

```
ServletConfig config = getServletConfig();
ServletContext context = getServletContext();
context.setAttribute("appName", "ShopSphere");
```

15. Sessions, Cookies, Listeners and Filters

Session

```
HttpSession session = request.getSession();
session.setAttribute("user", user);
session.setAttribute("role", user.getRole());
```

Use sessions for authentication state and cart state.

Cookies

Use cookies for non-sensitive convenience features such as recently viewed products. Never store plain passwords in cookies.

Authentication Filter

```
Request
  |
AuthenticationFilter
  |
Logged in?
 / Yes No
 |   |
Page Login
```

Admin Filter

Protect /admin/* and allow only users with the ADMIN role.

Event Handling

```
@WebListener
public class AppListener
       implements ServletContextListener {

    public void contextInitialized(
           ServletContextEvent event) {
        // application startup
    }

    public void contextDestroyed(
           ServletContextEvent event) {
```

```
        // application shutdown
    }
}
```

16. JSP Requirements

Students should understand JSP architecture and lifecycle, demonstrate traditional JSP elements for the syllabus, and use EL/JSTL for the main application.

```
<%@ page contentType="text/html;charset=UTF-8" %>
<h2>${product.name}</h2>
<p>Price: ₹${product.price}</p>
```

Traditional syntax to demonstrate:

```
<%! int count = 0; %>
<% String name = "Student"; %>
<%= name %>
<%-- JSP comment --%>
```

17. JSP Fragments, Tag Files and JSTL

```
fragments/
    header.jsp
    navbar.jsp
    footer.jsp
    sidebar.jsp

WEB-INF/tags/
    productCard.tag
    rating.tag

<%@ include file="fragments/header.jsp" %>

<c:forEach var="product" items="${products}">
    <h3>${product.name}</h3>
    <p>₹${product.price}</p>
</c:forEach>

<c:if test="${product.stock > 0}">
    <button>Add to Cart</button>
</c:if>
```

Use JSTL Functions for string/list operations. Demonstrate JSTL XML and SQL in separate syllabus-focused pages; keep the main architecture as JSP → Servlet → Service → DAO → JDBC.

18. RMI Module

Implement a remote inventory service.

```
public interface InventoryService extends Remote {
```

```
    Product getProduct(int id) throws RemoteException;
    int getStock(int productId) throws RemoteException;
    boolean updateStock(int productId, int quantity)
        throws RemoteException;
}
```

```
RMI Registry
    |
InventoryService
    |
Remote Implementation
    |
MySQL
```

19. Java Networking Module

Create a small inventory client-server program using java.net.

```
Server:
ServerSocket
   -> accept()
   -> Socket
   -> read command
   -> process/query
   -> send response

Client:
Socket
   -> send command
   -> receive response
```

Suggested commands: GET_PRODUCT, GET_STOCK, SEARCH_PRODUCT.

20. Serialization

```
public class Product implements Serializable {
    private int productId;
    private String name;
    private double price;
    private int stock;
}
```

Use ObjectOutputStream/ObjectInputStream in the networking demonstration to transfer Product or Order objects.

21. Swing Admin Application

Create a desktop admin application with a login window and management screens.

```
LoginFrame
  |
```

```
AdminDashboard
   |-- ProductFrame
   |-- CategoryFrame
   |-- CustomerFrame
   |-- OrderFrame
   `-- InventoryFrame
```

Use JFrame, JPanel, JLabel, JTextField, JPasswordField, JButton, JCheckBox, JRadioButton, JComboBox, JTable, JScrollPane and JOptionPane.

Swing MVC

```
Model      -> Product, User, Order
View       -> ProductFrame, LoginFrame
Controller -> ProductController, LoginController
DAO        -> ProductDAO, UserDAO
```

Pluggable Look and Feel

```
UIManager.setLookAndFeel(
    UIManager.getSystemLookAndFeelClassName()
);
```

22. J2EE and N-Tier Demonstration

The final viva should explain how the project uses a web container, HTTP, presentation layer, controller layer, service layer, DAO layer and database. Apache Tomcat is the web container used to deploy the WAR application.

23. Internationalization

```
messages_en.properties
messages_hi.properties
```

Demonstrate Locale, ResourceBundle and formatted prices/dates.

24. JNDI and URL/Protocol Demonstrations

Include a small JNDI resource-lookup demonstration and a separate URL/URLConnection content/protocol demonstration. These should remain small educational modules rather than complicating the main shopping workflow.

25. Modern E-Commerce UI

- Responsive navbar with search
- Product cards with images, discounts and ratings

- Cart icon and counter
- Responsive forms/tables
- Bootstrap layout
- Admin dashboard cards
- Order-status badges/timeline
- Success/error alerts
- Mobile-friendly pages

26. Security Requirements

- Use PreparedStatement.
- Hash passwords before database storage.
- Use HttpSession for login state.
- Use AuthenticationFilter for protected customer pages.
- Use AdminFilter for /admin/*.
- Validate input on the server.
- Calculate order totals on the server; do not trust browser-submitted totals.
- Never store passwords in cookies.

27. Maven / Tomcat Requirements

```
<packaging>war</packaging>

<dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.1.0</version>
    <scope>provided</scope>
</dependency>
```

Add MySQL Connector/J and the required JSTL API/implementation dependencies. For Tomcat 11, use jakarta.servlet.* rather than javax.servlet.*.

28. Development Plan for Students

Phase	Work	Output

1	Requirements + UI design	Use cases, screens, module list
2	Database	SQL script and ER diagram
3	Models + JDBC	POJOs and DAO CRUD
4	MVC/service	Controller/service/DAO structure
5	Authentication	Register/login/logout/session
6	Products	Listing/search/filter/details
7	Cart	Add/update/remove
8	Checkout	Address/coupon/order
9	Admin	Products/categories/inventory/orders
10	Servlet/JSP	Complete web interface
11	Filters/listeners	Security and events
12	JSTL/tags	Dynamic JSP and reusable UI
13	Networking	Socket client/server
14	RMI	Remote inventory service
15	Swing	Desktop admin module
16	Testing	Functional/integration tests
17	Report	Documentation/screenshots/viva

29. Minimum Working Version

If time is limited, students should complete the core application first:

- Database
- Registration/login
- Products
- Cart
- Checkout
- Order placement
- Admin product management

- Servlet + JSP + JDBC + MVC
- Session + Filter
- EL + JSTL
- Tomcat deployment

Only after the core flow works should students add RMI, Socket programming, Swing, JNDI and the additional syllabus demonstrations.

30. Testing Checklist

Test	Expected result
Valid registration	User created
Duplicate email	Validation/error
Valid login	Session created
Invalid login	Error shown
Unauthenticated cart access	Redirect to login
Search product	Matching products
Add to cart	Cart updated
Change quantity	Totals recalculated
Place order	Order and items created
Admin login	Dashboard opened
Customer opens /admin	Access denied
Admin updates order	Customer sees new status
Logout	Session invalidated
RMI call	Remote response
Socket request	Server response

31. Student Deliverables

- Complete source code
- pom.xml

- shopsphere.sql
- ER diagram
- System architecture diagram
- Use-case diagram
- Class diagram
- Login/order sequence diagrams
- Screenshots
- Test cases
- Setup guide
- Final project report
- Viva preparation

32. Suggested Project Report

1. Introduction
2. Problem Statement
3. Objectives
4. Requirements
5. Technology Stack
6. System Architecture
7. Database Design
8. Module Design
9. Implementation
10. Testing
11. Screenshots
12. RTU Syllabus Mapping
13. Limitations and Future Scope
14. Conclusion

References

33. Viva Demonstration Checklist

- Explain Swing MVC and show Swing components.
- Show JDBC CRUD.
- Run Socket client-server communication.
- Run RMI registry and remote call.
- Explain J2EE/N-tier architecture.

- Deploy the web application on Tomcat.
- Explain Servlet lifecycle.
- Demonstrate session and cookies.
- Demonstrate listeners and filters.
- Demonstrate JSP, EL and JSTL.
- Show fragments/tag files.
- Explain serialization, internationalization and JNDI.
- Show database and architecture diagrams.
- Explain where each RTU syllabus topic appears in the project.

34. Conclusion

ShopSphere is a single capstone project that connects the supplied RTU syllabus with a modern E-Commerce use case. Students should first build the core web application end-to-end and then add the Swing, networking, RMI and syllabus-specific JSP demonstrations. This approach keeps the project achievable while ensuring that the final demonstration covers the major Advanced Java topics.